

Projektowanie aplikacji mobilnych

Ćwiczenia #4 – Wstrzykiwanie dependencji i przechowywanie danych (DI and data persistence)

Grupa:

Ilias Abdeljalil (40028)

1. Zaimplementować framework do wstrzykiwania danych i wstrzyknąć przykładowy obiekt (2pkt)

Można wykorzystać bibliotekę tak jak w przykładach (Koin / Swinject lub inne) lub spróbować to zrobić samemu. Istotne, żeby przygotować aplikację na wstrzykiwanie różnych obiektów.

Screensy potwierdzające implementację:

```
1 using Assets.App.Code.MVVM.Models;
2 using Assets.App.Code.MVVM.View;
3 using Assets.App.Code.MVVM.ViewModels;
4 using UnityEngine;
5
6 namespace App.Code
7 {
8     [1 asset usage]
9     public class Container : MonoBehaviour
10     {
11         [SerializeField] private GameObject listItem; [1 asset usage]
12         [SerializeField] private ArchiveView archiveView; [1 asset usage]
13
14         private ArchiveModel archiveModel;
15         private ArchiveViewModel archiveViewModel;
16
17         [Event function]
18         private void Awake()
19         {
20             archiveModel = new ArchiveModel();
21             archiveViewModel = new ArchiveViewModel(archiveModel, archiveView, listItem);
22             archiveView.Construct(archiveModel, archiveViewModel);
23         }
24
25         [1 asset usage]
26         public void AddTextDocument() => archiveViewModel.AddTextDocument();
27
28         [1 asset usage]
29         public void AddListDocument() => archiveViewModel.AddListDocument();
30
31         [1 asset usage]
32         public void AddListItem() => archiveViewModel.AddListItem();
33
34         [1 asset usage]
35         public void RemoveListItem() => archiveViewModel.RemoveListItem();
36     }
37 }
```

```
namespace Assets.App.Code.MVVM.Models
{
    [2 usages] [dovara41 *]
    public class ArchiveModel
    {
        [4 usages]
        public List<Document> Documents { get; set; } = new();
    }

    [2 usages] [dovara41 +1]
    public class TextDocument : Document
    {
        [1 usage]
        public string Text { get; set; }
    }

    [2 usages] [dovara41 +1]
    public class ListDocument : Document
    {
        [1 usage]
        public List<string> Elements { get; set; }
    }

    [5 usages] [2 inheritors] [dovara41 +1]
    public class Document
    {
        [3 usages]
        public string Title { get; set; }
    }
}
```

2. Zaimplementować wstrzykiwanie ViewModeli w aplikacji za pomocą przygotowanego frameworku (2pkt)

Wszystkie dostępne w aplikacji ViewModele mają być zdefiniowane/zarejestrowane w module/obiekcie do wstrzykiwania. Następnie tam, gdzie są potrzebne, mają być wstrzyknięte.

Screeny potwierdzające poprawną implementację:

```
using Assets.App.Code.MVVM.Models;
using Assets.App.Code.MVVM.View;
using Assets.App.Code.MVVM.ViewModels;
using UnityEngine;

namespace App.Code
{
    // 1 asset usage
    public class Container : MonoBehaviour
    {
        [SerializeField] private GameObject listItem; // ListItem
        [Space]
        [SerializeField] private ArchiveView archiveView; // Canvas (ArchiveView)

        private ArchiveModel archiveModel;
        private ArchiveViewModel archiveViewModel;

        // Event function
        private void Awake()
        {
            archiveModel = new ArchiveModel();
            archiveViewModel = new ArchiveViewModel(archiveModel, archiveView, listItem);
            archiveView.Construct(archiveModel, archiveViewModel);
        }

        // 1 asset usage
        public void AddTextDocument() => archiveViewModel.AddTextDocument();

        // 1 asset usage
        public void AddListDocument() => archiveViewModel.AddListDocument();

        // 1 asset usage
        public void AddListItem() => archiveViewModel.AddListItem();

        // 1 asset usage
        public void RemoveListItem() => archiveViewModel.RemoveListItem();
    }
}
```

3. Zdefiniować interfejs/protokół lokalnego magazynu danych oraz zaimplementować go za pomocą SharedPreferences (android) lub UserDefaults (iOS) (2pkt)

Efektom tego zadania ma być posiadanie w aplikacji mechanizmu do przechowywania danych pomiędzy sesjami aplikacji. Dane mogą być dopasowane do Państwa aplikacji. Oczywiście musi istnieć możliwość zmiany tych danych używając aplikacji oraz sprawdzenia, że po ponownym włączeniu aplikacji, są one nadal poprawnie zapisane.

Screeny potwierdzające poprawną implementację: